

1. Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

```
<!DOCTYPE html>
<html>
<head>
  <title>Mock Zomato Admin Login</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f5f5f5;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }

    .login-box {
      background: white;
      padding: 30px;
      width: 300px;
      border-radius: 10px;
      box-shadow: 0 0 10px #ccc;
    }

    h2 {
      text-align: center;
    }

    input {
      width: 100%;
      padding: 10px;
      margin: 8px 0;
      box-sizing: border-box;
    }

    button {
      width: 100%;
      padding: 10px;
      background: #e23744;
      color: white;
      border: none;
      cursor: pointer;
    }

    #message {
      text-align: center;
```

```
        margin-top: 15px;
        font-weight: bold;
    }
</style>
</head>

<body>

<div class="login-box">
    <h2>Zomato Admin Panel</h2>

    <input type="text" id="username" placeholder="Username">
    <input type="password" id="password" placeholder="Password">

    <button onclick="login()">Login</button>

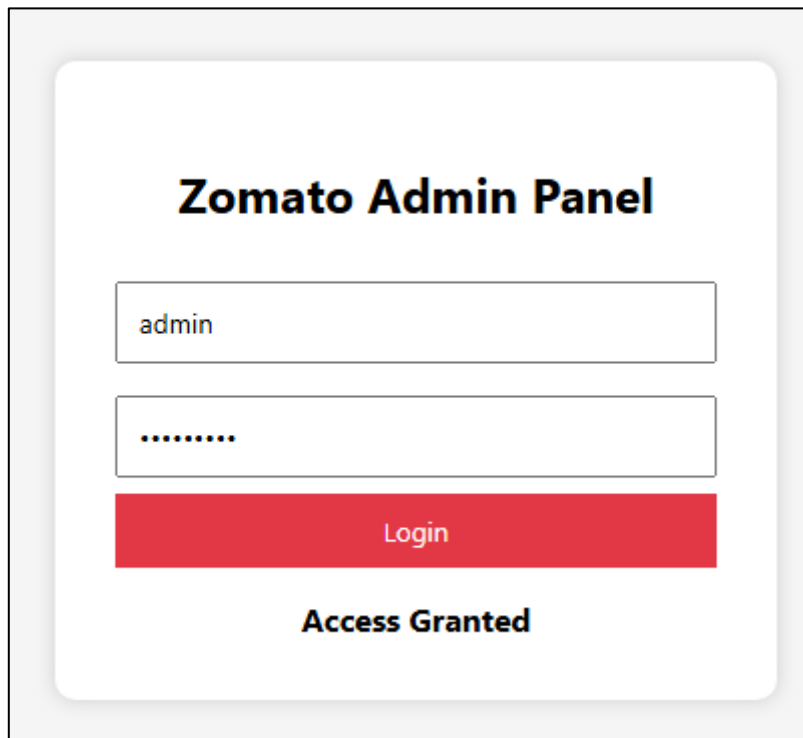
    <div id="message"></div>
</div>

<script>
function login() {
    const username = document.getElementById("username").value;
    const password = document.getElementById("password").value;

    // Hardcoded demo credentials
    const adminUsername = "admin";
    const adminPassword = "zomato123";

    if (username === adminUsername && password === adminPassword) {
        document.getElementById("message").textContent = "Access Granted";
    } else {
        document.getElementById("message").textContent = "Access Denied";
    }
}
</script>

</body>
</html>
```



2. Write a Python function `check_access(user_role, resource)` that returns True if the `user_role` is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'

```
def check_access(user_role, resource):  
    permissions = {  
        "customer": {  
            "order_history"  
        },  
        "restaurant_owner": {  
            "menu_edit",  
            "order_history"  
        },  
        "admin": {  
            "menu_edit",  
            "order_history",  
            "admin_dashboard"  
        }  
    }  
}
```

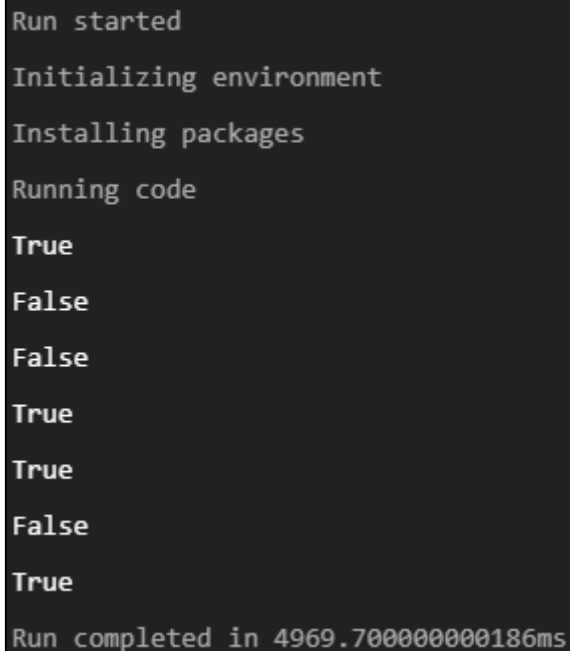
```
return resource in permissions.get(user_role, set())
```

```
# Test cases
```

```
print(check_access("customer", "order_history"))    # True  
print(check_access("customer", "menu_edit"))        # False  
print(check_access("customer", "admin_dashboard"))  # False
```

```
print(check_access("restaurant_owner", "menu_edit")) # True  
print(check_access("restaurant_owner", "order_history")) # True  
print(check_access("restaurant_owner", "admin_dashboard")) # False
```

```
print(check_access("admin", "admin_dashboard"))      # True
```



```
Run started  
Initializing environment  
Installing packages  
Running code  
True  
False  
False  
True  
True  
False  
True  
Run completed in 4969.700000000186ms
```

3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

```
import random
```

```
# Hardcoded credentials for simulation
```

```
CORRECT_USERNAME = "user123"
CORRECT_PASSWORD = "pass123"
```

```
def generate_otp():
    """Generate a random 6-digit OTP."""
    return str(random.randint(100000, 999999))
```

```
def login():
    print("=== Login ===")
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()

    if username != CORRECT_USERNAME or password != CORRECT_PASSWORD:
        print("Invalid username or password.")
        return
```

```
    print("Password verified. Proceeding to OTP verification...")
```

```
    # Step 2: Generate and "send" OTP (displayed in console to simulate an SMS/app)
    otp = generate_otp()
    print(f"\n[SIMULATED SMS] Your OTP is: {otp}\n")
```

```
    # Give the user 3 attempts, like most real-world systems
    max_attempts = 3
    for attempt in range(1, max_attempts + 1):
        entered_otp = input(f"Enter the 6-digit OTP (attempt {attempt}/{max_attempts}):")
        entered_otp = entered_otp.strip()
```

```
        if entered_otp == otp:
            print("\nLogin Successful")
            return
        else:
            print("Incorrect OTP.")
```

```
    print("\nToo many failed attempts. Login failed.")
```

```
if __name__ == "__main__":
    login()
```

```
=== Login ===
Enter username: user123
Enter password: password123
Password verified successfully.

[DEBUG] Your OTP is: 856066
An OTP has been sent to your registered mobile number.
Enter the 6-digit OTP:
```

4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

```
function getPremiumRecentUsers(users) {  
  const sevenDaysAgo = new Date();  
  sevenDaysAgo.setDate(sevenDaysAgo.getDate() - 7);  
  
  return users.filter(user =>  
    user.isPremium &&  
    new Date(user.lastLogin) >= sevenDaysAgo  
  );  
}
```

// Example

```
const users = [  
  { username: "Rahul", isPremium: true, lastLogin: "2026-08-08" },  
  { username: "Priya", isPremium: false, lastLogin: "2026-08-07" },  
  { username: "Amit", isPremium: true, lastLogin: "2026-07-25" },  
  { username: "Neha", isPremium: true, lastLogin: "2026-08-06" }  
];
```

```
console.log(getPremiumRecentUsers(users));
```

5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

5 Common Access Control Mistakes in Real-World Apps

1. Broken Object-Level Authorization (IDOR)

- Mistake: Users can access another user's data by changing an ID in the URL or request.
- Fix: Check server-side ownership and permissions for every object request.

2. Relying on Client-Side Checks

- Mistake: Developers hide buttons or pages but do not enforce permissions on the server.

- Fix: Always enforce authorization on the backend, not just in the app interface.

3. Excessive User Privileges

- Mistake: Users receive more permissions than they actually need.
- Fix: Apply the principle of least privilege and use role-based access control (RBAC).

4. Missing Authorization on APIs

- Mistake: Some sensitive API endpoints do not properly check the user's permissions.
- Fix: Protect every sensitive endpoint with centralized server-side authorization.

5. Weak Role and Permission Changes

- Mistake: Users can improperly change roles or gain higher privileges.
- Fix: Validate privilege changes on the server and keep audit logs of permission changes.